

ARK-451 — Authority Revocation During Execution

PREREGISTRATION (locked before execution)

Experiment ID: ARK-451

Series: ExecutionProof authorization-boundary corpus (enterprise-failure-mode phase)

Substrate: Classical software (no quantum hardware, no cryptography)

Monitors: Dual independent — V1 (JavaScript, no deps), V2 (Python)

1. Question

Permission at verification time is not permission at execution time. When authority is valid when a decision is made and a ProofRecord is bound, but is **revoked during the execution window**, does an independent execution monitor **fail closed** (DENY) or **fail safe** (HOLD) — never silently proceeding as if the authority were still valid?

This extends ARK-449 (state change after verification) to the specific, high-consequence case of **authority revocation** mid-flight: compromised credentials, employee termination, API-key rotation, emergency shutdown, and policy change.

2. Hypothesis

An execution monitor that re-checks authority at the moment of resource contact will:

- **ALLOW** only when authority is confirmed valid at execution (never revoked, revoked only after contact, or revoked-then-validly-reauthorized before contact);
 - **DENY** when authority is provably revoked (fully effective) before resource contact;
 - **HOLD** when a revocation is in-flight (issued but not yet fully propagated) at the moment of contact, because validity cannot be confirmed.
-

3. Model

Each scenario describes one authorization lifecycle on a monotonic time axis (seconds):

- `t_decision` — the authorization decision is made
- `t_bind` — the ProofRecord is bound ($t_decision \leq t_bind \leq t_execution$)
- `t_execution` — the action contacts the resource (the moment of execution)
- `multistep` — whether the action is the irreversible final step of a multi-step workflow
- `revocation` — `null`, or `{ t_revoke, propagation_delay, reason }`
- `reauthorization` — `null`, or `{ t_reauth, valid }`

A revocation becomes **fully effective** at $\text{eff} = t_{\text{revoke}} + \text{propagation_delay}$. The `propagation_delay` models the real-world gap between when a revocation is issued and when it is observable at the enforcement point.

4. Decision procedure (both monitors, identical)

Evaluated at `t_execution` (re-check at moment of resource contact):

1. If `revocation` is `null` → **ALLOW** (authority valid throughout).
 2. Compute $\text{eff} = t_{\text{revoke}} + \text{propagation_delay}$.
 3. If a `reauthorization` exists, is `valid`, and $t_{\text{revoke}} < t_{\text{reauth}} \leq t_{\text{execution}}$ → **ALLOW** (a new, independent decision governs).
 4. Else if $\text{eff} \leq t_{\text{execution}}$ → **DENY** (authority provably revoked before contact; fail closed).
 5. Else if $t_{\text{revoke}} \leq t_{\text{execution}} < \text{eff}$ → **HOLD** (revocation in-flight / unconfirmed at contact; fail safe).
 6. Else ($t_{\text{revoke}} > t_{\text{execution}}$) → **ALLOW** (revocation issued only after the action already contacted the resource under valid authority).
-

5. Arms

8 arms × 100 scenarios per arm = **800 evaluation decisions**.

Arm	Label	Timeline	Expected
1	VALID-throughout	no revocation	ALLOW
2	REVOKED-before-bind	revocation effective before <code>t_bind</code>	DENY
3	REVOKED-after-decision-before-contact	revoke after bind, effective before <code>t_execution</code>	DENY
4	REVOKED-during-multistep	multi-step; effective before the irreversible step	DENY
5	IN-FLIGHT-at-contact	issued before contact, not yet effective at <code>t_execution</code>	HOLD
6	REVOKED-then-REAUTHORIZED	revoked, then a valid reauth before <code>t_execution</code>	ALLOW
7	IN-FLIGHT-boundary	revoke very close to contact, still within propagation window	HOLD
8	REVOKED-after-execution	revocation issued strictly after <code>t_execution</code>	ALLOW

Randomized per scenario: `requester_id`, `action_type`, `resource`, all timeline offsets, `propagation_delay`, and revocation `reason` — subject to the arm's timing constraint.

6. Revocation-timing gate (anti-no-op oracle)

Carried forward from ARK-455b / ARK-454 / ARK-453. An **independent structural oracle** (`revocation_effective`) verifies, for every generated scenario, that it genuinely encodes its arm's timing relationship before any monitor sees it:

- Arm 1: `revocation is null`.
- Arm 2: `eff < t_bind` (effective before bind), no reauth.
- Arm 3: `t_bind ≤ t_revoke` and `eff ≤ t_execution`, no reauth.
- Arm 4: `multistep == true` and `eff ≤ t_execution`, no reauth.
- Arms 5, 7: `t_revoke ≤ t_execution < eff` (genuinely in-flight), no reauth.
- Arm 6: valid reauth with `t_revoke < t_reauth ≤ t_execution`.
- Arm 8: `t_revoke > t_execution` (genuinely after contact), no reauth.

If **any** scenario fails its check, the run **aborts** (exit 1). This prevents ARK-455-style defects where a case appears to encode a condition but is mathematically inert (e.g., a “revoked” arm whose revocation actually lands after execution).

7. Kill-gate (pre-corpus calibration)

`run_killgate.py` generates 100 calibration scenarios cycling through all 8 arms, runs the revocation-timing gate on every one, then evaluates with both monitors. **V1-V2 concordance must be $\geq 99\%$** to proceed to the full corpus; otherwise the run aborts.

8. Metrics

- **g_allow_valid_min** = minimum ALLOW rate across arms {1, 6, 8}
- **g_deny_revoked_min** = minimum DENY rate across arms {2, 3, 4}
- **g_hold_inflight_min** = minimum HOLD rate across arms {5, 7}
- **g_margin** = $\min(\text{g_allow_valid_min}, \text{g_deny_revoked_min}, \text{g_hold_inflight_min}) - 0.05$

9. Pass criteria (preregistered)

- **C1:** $\text{g_allow_valid_min} \geq 0.95$
- **C2:** $\text{g_deny_revoked_min} \geq 0.95$
- **C3:** $\text{g_hold_inflight_min} \geq 0.95$
- **C4:** $\text{g_margin} \geq 0.90$

Verdict: PASS if $C1 \wedge C2 \wedge C3 \wedge C4$, else FAIL. The recorded verdict stands as executed; no post-hoc criterion changes.

10. What a PASS supports / what a FAIL would mean

- **PASS:** The three-state model (ALLOW / HOLD / DENY) correctly handles revocation during the execution window — revoked authority fails closed, in-flight revocation fails safe, and only confirmed-valid (or validly re-authorized) authority proceeds. Supports the “verification before execution, continuously” claim.
- **FAIL:** Any arm where revoked authority is silently ALLOWed, or valid/after-contact authority is wrongly blocked, would falsify the claim and stand as an honest negative result (cf. ARK-445, ARK-455).

11. Reproducibility

- Deterministic seeding: `SEED_BASE = 20260717451` ; arm *k* uses `SEED_BASE + k` ; kill-gate uses `SEED_BASE + 99` .
- No external dependencies: V1 is pure Node.js, V2 is pure Python 3 stdlib.

- Re-running the scripts reproduces identical decisions and verdict (only wall-clock timestamps in the result JSON differ).

12. Provenance & discipline

1. Preregistration, schema, dual monitors, generator, run scripts, and MANIFEST (SHA-256 hashes) are committed **BEFORE** any scenario is generated or evaluated (LOCK).
2. Results are committed **AFTER** execution.
3. No post-hoc criterion changes; no rescue-after-failure; honest reporting of any FAIL.
4. Commits are not cryptographically signed — provenance = commit history + MANIFEST SHA-256 hashes.

Trademarks

ExecutionProof™, ProofRecord™, VaultProof™, Verification Before Execution™, Proof Before Power™ — Remnant Fieldworks Inc.

If it cannot be verified, it cannot execute.